



# Python for Fixed Income

COURSE 3 · MARKET DATA AND TIME SERIES

## Working with an AI Assistant

WEEK 3 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# Working with an AI assistant

Assistants write data code in seconds, and it usually runs. Whether it is right is your job. No AI tool is used today: every answer shown was written for this course in the style of an AI assistant's answer.

## Never paste or upload, into an AI tool or Colab

Client names, or anything that identifies a client

Positions, holdings, trades, or orders from a real book

Anything from an employer: code, data, screenshots, paths

Credentials: an API key, a password, a token

A URL or an error message with a key in it

**Your firm's policy comes first: whether, which assistant, and on what.**

## Today, in seven steps

13.1 The prompt is a spec

13.2 Pandas answers, reviewed

13.3 Chart code, checked

13.4 Bond code against bondmath

13.5 Debugging, safely

13.6 A package that does not exist

13.7 JSON replies

# The prompt is a spec

## 1. Docstring

What comes back, the units, the convention, what it raises.

## 2. Conventions

The rows of the course's conventions table that apply.

## 3. A test

The numbers the answer must reproduce, on dates that matter.

```
prompt = "\n\n".join(["Write a Python function with pandas and nothing else.",  
    "The docstring:\n" + DOCSTRING,  
    "Follow these conventions:\n" + CONVENTIONS,  
    "It must pass this test:\n" + TEST])
```

**An answer that says it passes your test has not passed it until you run it.**

# Three answers, each run against a test

| Answer | Asked for                      | Tested against             | Result               |
|--------|--------------------------------|----------------------------|----------------------|
| A      | Month-end levels, own dates    | md.last_per_period         | 4 of 10 labels wrong |
| B      | 2s10s and its daily change, bp | Excel; slope_bp, change_bp | Passes: gap 0 bp     |
| C      | 21-row vol, min_periods 21     | Excel; md.rolling_vol_bp   | 16 partial windows   |

A: `resample("ME").last()` labels January the 31st, a Saturday. September ends on a market day, so a test on September alone passes.

C: matches Excel's STDEV.S on all of 2026, and fails at the start of the history. Test the edges, not just the middle.

**All three ran without an error. Running is not the test.**

# Chart code: check it against the numbers

```
# the assistant's chart (written for the course)
slope = (curve["10 Yr"] - curve["2 Yr"]) * 100
ax.plot(slope.index, slope.to_numpy())
ax.set_ylabel("percent")
```

```
assert unit in ax.get_ylabel()
plotted = ax.lines[0].get_ydata()
assert (abs(plotted - series) <= 1e-9).all()
```

## What the check found

The line: exactly your slope\_bp. Right.

**The axis: says percent, the data is in bp. Wrong.**

The last point, 45, would read as 45 percent. Fixed: 45.0 bp.

A chart cannot raise an error for a wrong word.

**Read the chart object: its labels and the numbers its lines hold. Looks right becomes a test.**

# An assistant's DV01 against your own bondmath

| Issuer (synthetic) | Excel and bondmath | Assistant | Low by |
|--------------------|--------------------|-----------|--------|
| Quorvane Energy    | 0.041903           | 0.041537  | 0.87%  |
| Dunmarrow Rail     | 0.055175           | 0.054618  | 1.01%  |
| Pellwick Paper     | 0.026359           | 0.026152  | 0.78%  |
| Halvering Foods    | 0.067511           | 0.066666  | 1.25%  |

Course 2's running bonds, settled 2026-11-30, between coupons. DV01 per 100.

```
duration * bondmath.price(...) * 0.0001 # clean, not dirty
```

Choose test cases where the mistake would show.

On a coupon date nothing has accrued: clean equals dirty, and the bug hides.

# Debugging with an assistant

## Paste

The line of code that failed

The error's type and message

A few rows of public or synthetic data

Your Python and pandas versions

## Never paste

A key, a password, a token

A URL or an error that holds a key

Clients, positions, trades, employer data

A full path with your user name

**raise\_for\_status's message, through redact and safe\_to\_paste:**

```
.../fred/series/observations?series_id=DGS10&api_key=***&file_type=json
```

A safe question carries the rule: change\_bp says "series has 1 missing values; drop them first with `.dropna()`", and a missing day may never be filled.

# A package that does not exist

**fredbond-tools: a name invented for this course. No such project on PyPI, checked 2026-10-04.**

## Why installing it is dangerous

Anyone can register an unused name on PyPI. Installing runs code from whoever owns the name, and importing runs it again, with your access to your files.

## Check first (installs nothing)

1. Look the name up on [pypi.org](https://pypi.org) in your browser.

```
python -m pip index versions <name>
ERROR: No matching distribution
found for fredbond-tools
```

An argument can be invented too: `inspect.signature(pd.Series.resample)` has no `how`.

# JSON replies and parse\_json\_reply

```
{"series_id": "DGS10", "units": "bp", "window_rows": 63, "min_periods": 63, ...}
```

| Reply (written for the course) | parse_json_reply                              |
|--------------------------------|-----------------------------------------------|
| clean                          | ok, 5 fields                                  |
| with a sentence                | not JSON: Expecting value at line 1, column 1 |
| missing a field                | the reply is missing min_periods              |

Three new tests: fields kept, missing fields named, prose refused. 51 passed.

**parse\_json\_reply checks the shape. Your conventions check the content: units bp, min\_periods equal to the window.**

# Excel side by side: a suggested formula, tested

## Excel side by side (tested in Excel)

|                                                                    |                               |
|--------------------------------------------------------------------|-------------------------------|
| <code>=COUNTIF(B1:B252,"&lt;="&amp;B252)/COUNT(B1:B252)*100</code> | 99.60, the course's measure   |
| <code>=PERCENTRANK.INC(B1:B252,B252)</code>                        | 0.996, the suggestion         |
| <code>=COUNTIF(A1:A10,"&lt;="&amp;7)/COUNT(A1:A10)</code>          | 0.5                           |
| <code>=PERCENTRANK.INC(A1:A10,7)</code>                            | 0.222                         |
| <code>=B4*(B1+B2)/10000</code>                                     | 0.04190, DV01 (Course 2)      |
| <code>=B4*B1/10000</code>                                          | 0.04154, the clean-price twin |

Probe set 5, 7, 7, 9, 11, 12, 3, 7, 10, 8 at 7: five of ten at or below. PERCENTRANK.INC counts the 2 strictly below over  $n - 1$  and cuts to 3 digits.

**Over the 190 reference rows of 2026 the suggestion differs on 172, by up to 5.02 points.**

Tested in Excel 16.0 build 20430, 2026-10-04.

# Save the day: reviewing an assistant's patch

## 1. Check

```
git apply --check
```

## 2. Apply

```
git apply
```

## 3. Read

```
git diff
```

## 4. Test

```
pytest
```

## 5. Undo

```
git restore
```

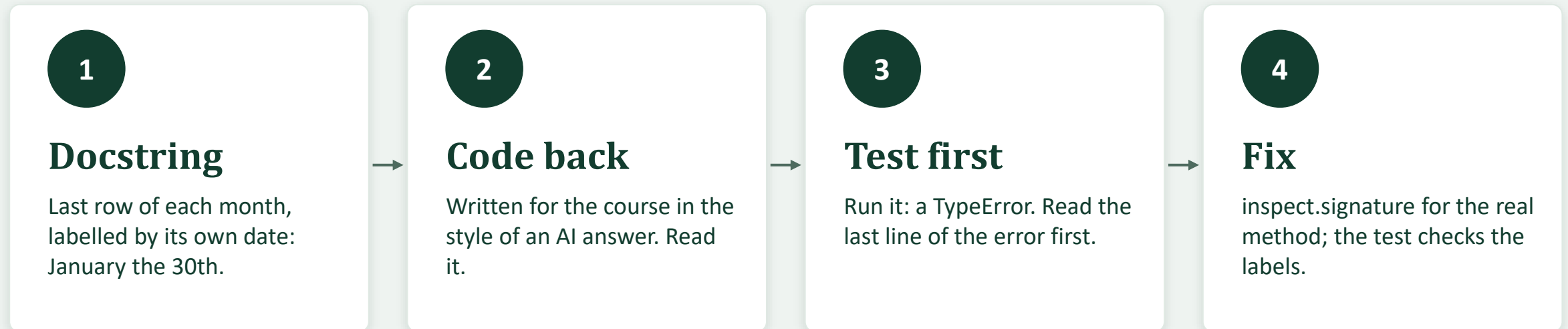
```
- # parse the reply; a reply with prose around the JSON fails here
+ start, end = text.find("{"), text.rfind("}")
-     reply = json.loads(text)
+     reply = json.loads(text[start:end + 1])
```

Tests on the patched module: 1 failed, 50 passed. test\_parse\_json\_reply\_not\_json\_raises: did not raise.

**The patch changed what the function promises. The test, written from the docstring, caught it. Restored.**

# The AI check

The docstring is the prompt. Today the code that comes back does not even run.



Today: `resample("ME", how="last")`. There is no `how`. And `resample("ME").last()` labels January the 31st.

*Code that runs is not the same as code that is right.*

# What you can do now

- Write a prompt as a spec: docstring, conventions, and a test
- Run every answer against your own functions and Excel, edges too
- Check a chart by its labels and the numbers its lines hold
- Paste nothing secret or from work; your firm's policy comes first
- Check a package on PyPI before installing; check JSON before using it

**Week 3, Day 4** The daily market snapshot: one function, any date, written to HTML and Excel